The Conscious Loop: Gated Continual Specialization of Small Open Models to Frontier-Level Accuracy at a Fraction of the Cost

TECHNICAL REPORT · 19 OCTOBER 2026

Arpit Sharma* **Tathagat Sharma***

UltraSafe AI

ABSTRACT

General-purpose frontier language models are the default choice for enterprise language tasks, yet most production workloads are narrow, repetitive and governed by an organization's own conventions. We present the Conscious Loop, the continual specialization method of the RunBios platform. The Loop turns an organization's labeled records into successive versions of a small open model. Each version is trained with a recipe derived from the size of its data, compared with the version in service on held-back records and on a fixed benchmark, and promoted only when the improvement is statistically supported. We evaluate the approach in two studies. In the first, we apply the Loop's specialize, evaluate and select cycle to five enterprise classification tasks built from real public records and compare the selected models of 8 to 27 billion parameters with Claude Opus 5.5, GPT-5.5 and GPT-6 Astra on 500 untouched test cases per task. The specialized models are significantly more accurate than the strongest frontier model on 3 tasks (+6.6, +8.6, +4.2 percentage points) and within sampling error of it on the other 2, while serving cost per request is 4.1 to 19.0 times lower. In the second study the Loop trains, compares and benchmarks successive versions on its own. Versions were put live when they beat the live version on the benchmark, and we report the held-back evidence behind each. We release the complete run log and every model answer.

Data, logs and code: github.com/RunBIOS-Research/gated-continual-specialization-of-small-open-models-to-frontier-level-accuracy

1 Introduction

A large share of enterprise language work consists of classification under a fixed label set. A bank routes customer messages to support teams, a lender files complaints under a regulator's taxonomy, a legal department labels contract clauses, a reporting team tags figures in financial filings and a security team assigns a weakness type to each new vulnerability. Each task runs at high volume, has an objectively correct answer and follows conventions that are specific to the organization or its regulator.

Frontier models perform these tasks well when the label set is supplied in the prompt. They are, however, billed per token on every request, they cannot be adapted to an organization's records without fine-tuning access, and their behavior changes on the provider's schedule rather than the organization's. A small open model fine-tuned on the organization's own records addresses all three

*Equal contribution. Correspondence: {arpit, tathagat}@us.inc

© 2026 UltraSafe AI. All rights reserved

concerns, provided it can be made as accurate as the frontier model and kept accurate as data and conventions evolve.

The Conscious Loop is designed for that second requirement. It treats specialization as a continual process rather than a single training run. New labeled records accumulate, a new version is trained, the new version is tested against the version in service and it replaces that version only when the evidence supports it. This report describes the method and evaluates it. Our contributions are the following.

1. A description of the Conscious Loop, including its step-aware training recipe and its statistically gated promotion rule (Section 3).
2. A controlled evaluation on five enterprise tasks built only from real public records, with frozen test sets, selection on separate validation data and exact-match scoring with no model-based judge (Sections 4 and 5).
3. Evidence that specialized open models of 8 to 27 billion parameters are significantly more accurate than the strongest of three frontier models on 3 of 5 tasks and within sampling error of it on the other 2, at 4.1 to 19.0 times lower serving cost (Section 5).
4. An analysis of what drives the gains, namely the volume of real training data, its recency and the compression of the instruction (Section 5.3).
5. Results from the Loop running continually on the same records, and a public evidence package that allows every reported number to be recomputed (Sections 5.4 and 8).

2 Related work

Continual learning and forgetting. Adapting a pretrained language model to a domain or task by further training on in-domain text is well established and reliably improves downstream accuracy [11]. Training the same model on a sequence of tasks is harder, because each new task can erase what earlier training taught. Scialom et al. [12] showed that instruction-tuned models can learn a stream of new tasks when a small fraction of earlier data is rehearsed alongside the new data, and Luo et al. [13] measured how much general ability is lost when models of 1 to 7 billion parameters are fine-tuned repeatedly without such care. Surveys of continual learning for language models organize the remedies into replay, regularization and parameter isolation [14]. Biderman et al. [15] found that low-rank adapters forget less than full fine-tuning, at the price of learning less on data that is far from the pretraining distribution. The Loop is a continual process for a single narrow task rather than a sequence of different tasks. It retrains from the base model or from the live version with a replayed sample of earlier records, and it measures each version on a fixed benchmark, so forgetting on the task of interest is detected and blocked before a version is promoted rather than estimated after the fact.

Parameter-efficient fine-tuning. Low-rank adaptation [8] trains a small number of added weights while the base model stays frozen, and quantized variants reduce the memory needed to fine-tune models of tens of billions of parameters on a single GPU [16]. These methods make it practical to train many versions of a model from modest amounts of data, which the Loop depends on. The Loop uses LoRA as its training primitive without modification. Its contribution lies in deriving the training recipe from the size of each attempt's data and in deciding, with a statistical test, which of the resulting versions goes into service.

Small fine-tuned models against frontier models. Several studies have compared fine-tuned small models with large general models used through prompting on classification tasks. Bucher and Martini [17] found that fine-tuned encoder models outperform GPT-3.5, GPT-4 and Claude Opus used zero-shot on sentiment, stance and emotion classification, Edwards and Camacho-Collados [18] reached the same conclusion for few-shot prompting against fine-tuning on 16 classification datasets, and Zhao et al. [19] reported that LoRA fine-tuned models of up to 7 billion parameters outperform GPT-4 by 10 points on average across 31 tasks. The cost of frontier inference has motivated cascades that route easy requests to cheaper models [20] and routers trained on preference data that send only hard queries to the strongest model [21]. Those studies take one trained model per task and compare it once. The Loop treats specialization as an ongoing process, and the present report adds a comparison with current frontier models on five enterprise tasks built from real records, with frozen test sets, selection on separate validation data and exact-match scoring.

Statistical comparison and gated promotion. Comparing two systems on the same test cases calls for a paired test, and guides for language technology recommend tests such as McNemar's [9] for paired accuracies together with a stated significance level [22]. Card et al. [23] showed that many published comparisons are underpowered for the differences they claim, and Miller [24] argued for reporting evaluation results with confidence intervals and paired analyses as a matter of routine. In production systems, a new model is usually put behind a champion and challenger comparison before it replaces the one in service, and surveys of deployment practice list monitoring, validation and safe rollout among the main difficulties of keeping a deployed model current [25]. The Loop applies these recommendations inside the training process rather than at publication time. Each candidate is compared with the version in service on paired held-back records, the result must be significant before promotion, and the test sets of this report are sized so that 95 percent intervals [10] are a few points wide.

3 The Conscious Loop

3.1 Setting

A task is defined by an instruction, a finite label set and a stream of labeled records, each pairing an input text with its correct label. The organization runs a live model that answers production requests. The Loop maintains three disjoint pools of records. Training records are used to fit new versions. A held-back set, carved from the same stream, is used to compare a new version with the live one. A fixed benchmark of 20 to 200 records is pinned once and replayed for every version, so that progress over time is measured on the same instrument.

3.2 The cycle

Figure 1 shows one cycle. Records arrive from production capture or from an import and are labeled by people or by the organization's own systems. When the pipeline's trigger fires the Loop curates a training set, removes duplicates, sets aside the held-back rows and trains a new version from the base model or from the live version. It then deploys the new version beside the live one, scores both on the held-back set and on the benchmark and either promotes the new version or keeps the live one. Every attempt, score and decision is recorded and visible on the RunBios dashboard.

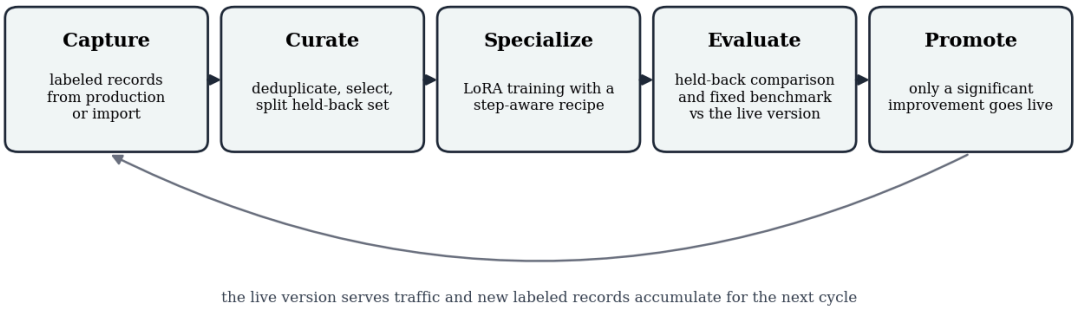


Figure 1. One cycle of the Conscious Loop. Each cycle ends with the live version either replaced by a better one or kept.

3.3 Step-aware training recipe

Each attempt fine-tunes a LoRA adapter on the base model. The recipe is not fixed. It is derived from the size of the attempt's data. The trainer packs short records into sequences, so a fixed number of epochs can yield only a few optimizer steps on a small set. The Loop therefore sets the sequence length from the measured length of the records, targets about 1,100 optimizer steps with between 3 and 15 passes over the data, keeps 5 percent of the training rows (at least 50 and at most 500) as a validation slice, evaluates every tenth of the planned steps, stops early after three evaluations without improvement and keeps the checkpoint with the best evaluated validation loss. The 1,100-step target was chosen from internal measurements on RunBios, where recall of held-out facts rose with the number of steps up to about that point and then stopped improving.

3.4 Gated promotion

A new version is compared with the live version on the held-back set. A judge model scores both answers against the reference and also compares them directly twice with the order of the two answers swapped. A record counts as a win only when both orders agree. The new version is classed as better only if it clears a minimum number of compared records, a bar on the mean score change and a win rate, and if a one-sided sign test on wins and losses is significant at the 5 percent level. The fixed benchmark is replayed for both versions on every attempt that completes its comparison. A benchmark can use exact matching, in which case no judge is involved. Promotion can be automatic or require a person's approval.

3.5 Building on earlier versions

When enabled, an attempt can train from the live version instead of from the base model. It then trains on the records added since that version, together with records from earlier attempts that did not go live and a replayed sample of earlier records. Whenever a record that an earlier version learned from is deleted, the next attempt rebuilds the model from the base on all remaining records.

Algorithm 1. One cycle of the Conscious Loop.

```
input   base model, live version, labeled records, held-back set, fixed benchmark
1  wait until enough new labeled records have accumulated
2  build the training set and remove duplicates and held-back records
3  derive the recipe from the set size: sequence length, passes, evaluation interval
4  train a LoRA adapter and keep the best evaluated checkpoint
5  deploy the new version beside the live version
6  score both on the held-back set and apply the sign test at 5 percent
7  replay the fixed benchmark for both versions
8  if the new version is better then promote it else keep the live version
9  record every score and decision and return to step 1
```

4 Experimental design

4.1 Two studies

We evaluate the method in two studies that share the same tasks and data. In Study A we execute the Loop's specialize, evaluate and select cycle through the RunBios training interface under researcher control, so that the volume and recency of the training data, the base model and the instruction format can be varied one at a time. Each round produces new versions, every version is scored on a held-out validation set and the version with the highest validation accuracy is retained. Study A trains with fixed epochs (2 to 3) rather than the step-aware recipe, and it gates on validation accuracy rather than on the judge-based comparison. In Study B the Conscious Loop runs on the same records with its own recipe, comparison and benchmark. It starts, trains and evaluates each attempt by itself, and versions were put live under the rule stated in Section 5.4.

4.2 Tasks and data

Each task uses real records from a public source. No synthetic or model-generated examples were used. Table 1 summarizes the data. The validation and test cases of each task were drawn before any training and never used for training.

Table 1. Tasks, sources and data sizes. Training examples are those used by the selected model of Study A.

Task	Source	License	Labels	Training examples	Validation	Test	Test cases drawn from
Banking intent routing	Banking77 (PolyAI)	CC-BY-4.0	77	9,968	300	500	official test split
Consumer-complaint triage	CFPB Consumer Complaint Database	CC0 (US government data)	45	15,000	286	500	complaints received January to February 2024 (after every training record)
Contract clause classification	LEDGAR from LexGLUE	CC-BY-4.0	100	66,737	300	500	official test split
XBRL numeric tagging	FiNER-139	CC-BY-SA-4.0	139	124,454	297	500	official test split
Vulnerability root-cause triage	NIST National Vulnerability Database	Public domain	54	34,906	300	500	CVEs published July to September 2026 (after every training record)

Complaint narratives come from a CC0 mirror of the CFPB database because the current official download no longer contains narratives. A random sample of 2,000 complaint IDs matched the official file on product and issue in every case. CFPB redactions (XXXX) were kept. CVE descriptions that name their own CWE identifier or the CWE's name were removed from training, validation and test data because they reveal the answer.

4.3 Baselines

The frontier baselines were anthropic/claude-opus-5-5, openai/gpt-5.5 and openai/gpt-6-astra, called through the RunBios serverless gateway with temperature 0. Each received the task instruction together with the complete list of allowed labels. The open base models were Llama-3.1-8B-Instruct, Qwen3.5-9B and Qwen3.6-27B, with Qwen3.6-35B-A3B tried on one task. The untrained Llama-3.1-8B-Instruct provides a lower reference.

4.4 Protocol

Contamination control. Test sets contain 500 cases per task and validation sets contain 286 to 300 cases. Training records whose normalized text matched any validation or test case were removed. A final check of every uploaded training row against the test cases found zero identical texts in all five tasks. For XBRL tagging the check was repeated at sentence level with the markers removed. For complaint triage and vulnerability triage every test case is later than every training record.

Instruction format. Fine-tuned models were trained and evaluated with the instruction they would receive in production. For some versions this was the same instruction as the baselines, including the label list. For others it was a single sentence without the list, which gives the model less information and shortens every request. The tables mark which format each version used.

Training. All Study A versions were trained with LoRA adapters through RunBios managed training in bf16 with a maximum sequence length of 2,048 tokens. Batch size, learning-rate schedule and GPU placement were set by the platform.

Metric and statistics. A response is correct only if it equals the reference label after lower-casing and collapsing whitespace. We report exact-match accuracy with 95 percent Wilson intervals and compare the selected model with the strongest frontier model by the exact two-sided McNemar test on the paired test cases. Test accuracy played no part in selection.

Cost. Frontier cost is the amount RunBios billed for the 500 test calls. The cost of a specialized model is the hourly price of its dedicated deployment divided by its measured throughput with 32 concurrent requests in an isolated run. Training cost is the amount billed for the training job.

5 Results

5.1 Specialized models against frontier models

Table 2 and Figure 2 report test accuracy. The selected specialized model is more accurate than every frontier model on 5 of 5 tasks. The margin over the strongest frontier model is significant at the 5 percent level for banking intent routing, consumer-complaint triage and contract clause classification. For XBRL numeric tagging and vulnerability root-cause triage the specialized model is ahead by a margin within sampling error, and we report these results as parity.

Table 2. Exact-match accuracy (%) on 500 test cases per task with 95% Wilson intervals. Difference is the selected model minus the strongest frontier model in percentage points. p is the exact McNemar test against that model. Untrained is Llama-3.1-8B-Instruct without fine-tuning.

Task	Untrained	RunBios fine-tuned	Claude Opus 5.5	GPT-5.5	GPT-6 Astra	Difference	p
Banking intent routing	51.8 [47.4, 56.1]	**95.0 [92.7, 96.6]** RunBios fine-tuned (Llama-3.1-8B)	88.4 [85.3, 90.9]	83.4 [79.9, 86.4]	86.0 [82.7, 88.8]	+6.6	< 0.001
Consumer-complaint triage	28.6 [24.8, 32.7]	**65.4 [61.1, 69.4]** RunBios fine-tuned (Llama-3.1-8B)	56.8 [52.4, 61.1]	54.0 [49.6, 58.3]	55.8 [51.4, 60.1]	+8.6	< 0.001
Contract clause classification	61.6 [57.3, 65.8]	**89.0 [86.0, 91.5]** RunBios fine-tuned (Qwen3.6-27B)	84.8 [81.4, 87.7]	79.8 [76.1, 83.1]	83.0 [79.5, 86.0]	+4.2	0.004
XBRL numeric tagging	58.6 [54.2, 62.8]	**87.4 [84.2, 90.0]** RunBios fine-tuned (Qwen3.5-9B)	86.4 [83.1, 89.1]	81.8 [78.2, 84.9]	80.2 [76.5, 83.5]	+1.0	0.551
Vulnerability root-cause triage	53.2 [48.8, 57.5]	**76.6 [72.7, 80.1]** RunBios fine-tuned (Qwen3.6-27B)	76.2 [72.3, 79.7]	67.0 [62.8, 71.0]	69.2 [65.0, 73.1]	+0.4	0.902

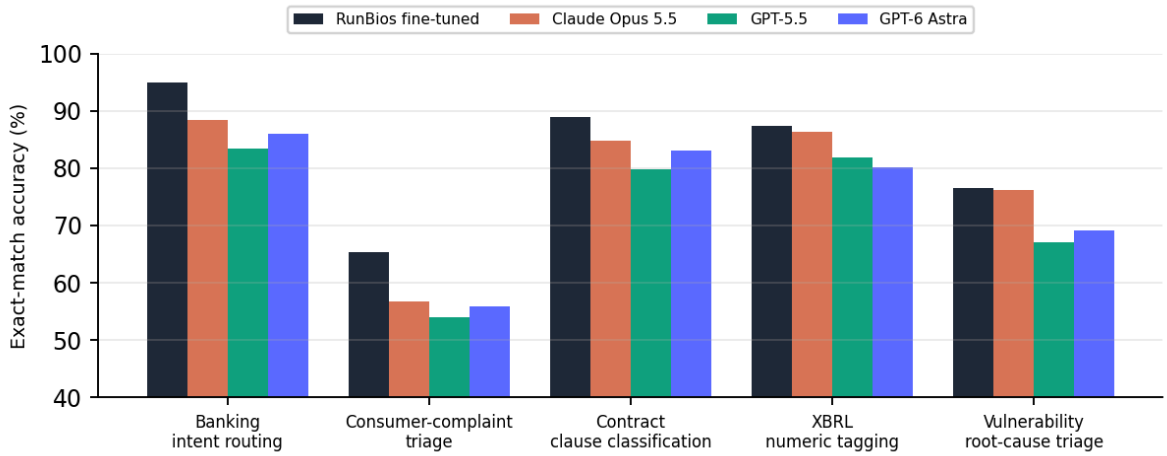


Figure 2. Test accuracy of the selected specialized model and three frontier models.

5.2 Cost

Table 3 and Figure 3 report cost. The specialized models cost between $0.08 and $0.61 per 1,000 requests, compared with $1.52 to $3.69 for the strongest frontier model. At these rates the one-time training cost of the selected model is recovered after between 5,640 and 162,737 requests, depending on the task.

Table 3. Cost per 1,000 requests in US dollars, the serving configuration of the selected model and the one-time cost of its training job.

Task	RunBios fine-tuned	Claude Opus 5.5	GPT-5.5	GPT-6 Astra	Saving vs best frontier	Serving	Training
Banking intent routing	$0.08	$1.52	$4.53	$5.89	19.0x	1x H100 NVL 94GB, $4.47/h, 15.6 req/s	$10.69
Consumer-complaint triage	$0.50	$3.69	$8.45	$9.86	7.3x	1x H100 NVL 94GB, $4.47/h, 2.5 req/s	$17.98
Contract clause classification	$0.25	$1.67	$4.09	$5.76	6.7x	1x H200 141GB, $7.41/h, 8.2 req/s	$72.53
XBRL numeric tagging	$0.61	$2.51	$10.84	$15.77	4.1x	1x H100 80GB, $5.59/h, 2.6 req/s	$192.86
Vulnerability root-cause triage	$0.41	$2.28	$8.76	$11.09	5.5x	1x H200 141GB, $7.41/h, 5.0 req/s	$302.88

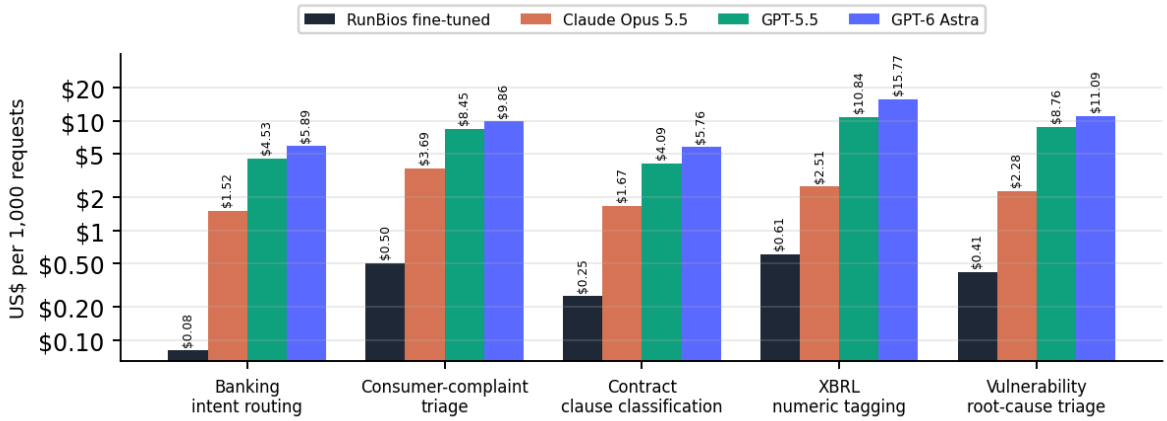


Figure 3. Serving cost in US dollars per 1,000 requests. The vertical axis is logarithmic so that costs from a few cents to over ten dollars remain readable, and each bar is labeled with its value.

5.3 What drives specialization

Volume of real data. The largest gains came from more real training records. On XBRL tagging, increasing the training set from 57,466 to 124,454 examples raised test accuracy from 80.6 to 87.4 percent. On contract classification, moving from 19,811 to 66,737 examples raised it from 85.4 to 89.0 percent. This mirrors how the Loop grows its training set as records accumulate in production.

Recency of data. On vulnerability triage, adding 15,554 CVEs published in the first half of 2026 raised test accuracy to 76.6 percent. These records follow the labeling conventions of the test period. On the 59 test cases labeled CWE-862 or CWE-863, the earlier Qwen3.6-27B version trained on 59,327 older records answered 20 correctly and gave the generic CWE-284 in 19 cases. The selected version answered 40 correctly and gave CWE-284 once. Conventions that drift over time are the situation continual specialization is designed for.

Instruction compression. A version trained with a one-sentence instruction no longer needs the label list at inference time. On contract classification this version was both the most accurate and the least expensive, at $0.25 per 1,000 requests.

Appendix A shows, for each task, the sequence of versions that led to the selected model.

5.4 The Loop running continually

In Study B a Conscious Loop pipeline was created for each task on the same records. Table 4 lists the benchmark score of the version that was live before the first promotion and the score of each version that was put live. Benchmarks contain 200 fixed cases, so these percentages are not directly comparable with the 500-case test results. During this study a version was promoted when it exceeded the live version on the benchmark, including cases where the held-back comparison did not reach significance. In an earlier pipeline on the banking task, the untrained Llama-3.1-8B scored 53.0 percent on the pipeline's benchmark, the first version put live scored 78.0 percent, and a later attempt trained on 2,168 records reached 84.5 percent but was not promoted because its held-back comparison did not show a significant improvement.

Table 4. Conscious Loop pipelines (Study B). Scores are exact-match accuracy (%) on each pipeline's 200-case benchmark.

Task	Base model	Starting live score	Promoted versions	Scored attempts	Pipeline ID
Banking intent routing	Llama-3.1-8B	48.5	90.0 → 91.5	3	`f30e21bb`
Consumer-complaint triage	Llama-3.1-8B	24.0	56.0 → 61.0	2	`7daee5f9`
Contract clause classification	Qwen3.6-27B	84.0	85.5 → 87.0	4	`b3330f78`
XBRL numeric tagging	Qwen3.5-9B	70.0	75.0 → 79.0	2	`ca430a27`
Vulnerability root-cause triage	Qwen3.6-27B	64.0	68.5 → 69.0 → 69.5	3	`f0685159`

6 Discussion

The results support three conclusions. First, for narrow classification work, a small open model specialized on an organization's real records can match or exceed the strongest frontier models. Second, in these experiments the gains followed data more than model size. The selected models range from 8 to 27 billion parameters, and the largest improvements followed increases in the volume or recency of real records. Third, the cost advantage is large and consistent, because each specialized model runs on a single dedicated GPU and, when trained with a short instruction, needs fewer prompt tokens.

These conclusions motivate the design of the Loop. If accuracy follows the volume and recency of real data, a model should be retrained as data accumulates, and each new version should replace the old one only on evidence. The gated promotion rule prevents regressions from reaching production, and the fixed benchmark makes improvement over time visible to the organization.

On XBRL tagging and vulnerability triage both the strongest frontier model and the specialized model fail on 7 to 16 percent of test cases. Many of these cases are ambiguous under the label set, which bounds the margin any model can reach on these tasks.

7 Limitations

- The main experiments ran on the RunBios pre-production environment between 13 and 16 October 2026, and the two reproduction runs ran on the production platform on 16 October 2026. Frontier model behavior and prices may change.
- Each configuration was trained once. Test sets of 500 cases give 95 percent intervals of roughly plus or minus 2 to 4 points.

- Study A applied the Loop's cycle under researcher control with fixed epochs and validation-based selection. Its results therefore measure the specialize, evaluate and select principle rather than every component of the full Loop, which Study B reports separately.
- Banking77, LEDGAR and FiNER-139 have been public since 2020 to 2022 and may appear in frontier pre-training data, which would favor the frontier models. Complaint and vulnerability test cases postdate all training data used here.
- A few validation cases share their text with a test case because the source contains duplicate records (28 for complaints, 7 for XBRL tagging and 3 for vulnerabilities). Validation was used only for selection, and no training row matches any test case.
- For banking routing and complaint triage no version was scored on validation. The reported model is the last version trained, which used the most data.
- Latency is not reported because measurement conditions differed between dedicated and serverless models.
- Gemma-4-31B could not be evaluated. Its adapter fine-tune trained but is not yet servable on RunBios, and full fine-tuning is not yet offered for it.

8 Data and code availability

The evidence package for this report is publicly available at https://github.com/RunBIOS-Research/gated-continual-specialization-of-small-open-models-to-frontier-level-accuracy (version 1.0.0). It contains the 500 test cases and the validation cases of every task, the exact training rows of every selected model, every answer produced by every model together with its token counts and billed cost, and the complete run log (Appendix B). It also contains the training logs of all 41 fine-tuning jobs (configuration, per-step training and validation metrics, checkpoints and evaluations), the serving and throughput records of every evaluated model, and for every Conscious Loop attempt of Study B the run record, the held-back comparison and the benchmark replay answer by answer. A standalone script recomputes every accuracy, interval, significance test and cost reported here from the package alone, using only the Python standard library, and a SHA-256 manifest covers every file.

The package also provides scripts that use only the public RunBios API to ask the frontier models the same test cases, to retrain, deploy and test each selected model from its released training rows, and to create and run a Conscious Loop pipeline on any task, together with a guide to every step of the Loop and its API calls. Reproduction is statistical rather than bit-exact, because fine-tuning is not deterministic and providers update frontier models, so a faithful reproduction is expected to fall within the reported 95 percent intervals. No RunBios platform source code is included or required.

To test the package itself, the selected banking-intent-routing configuration was retrained after the study on the public RunBios API, using only the released scripts and training rows. The retrained model scored 94.0 percent on the same 500 test cases, against the reported 95.0 percent and its 95 percent interval of 92.7 to 96.6 percent. Its training job cost $7.02.

A Conscious Loop pipeline for the same task was then created and run on the public RunBios API with the released script. On its 200-case benchmark the untrained model scored 47.5 percent. The first version, trained on 2,454 curated samples, scored 83.5 percent and was promoted on the Loop's own held-back verdict. The pipeline then started a second attempt by itself, built on the first version with 1,346 new and 1,346 replayed samples. It scored 89.0 percent and was promoted under the Study B rule, although its held-back comparison did not reach significance. Both re-runs, with every record, are included in the package. The repository also lists read-only public previews of these production runs on the RunBios platform, which anyone can open without an account. The package, not those pages, is the permanent record of every result in this report.

Every training job, deployment, dataset and Conscious Loop pipeline is identified in the package by its RunBios record ID. Tables in this report give the first eight characters of each ID. The records themselves are kept in the RunBios pre-production environment, and reviewers and auditors may request supervised access to them from UltraSafe AI under a confidentiality agreement. In total 7,500 frontier test calls were scored and $3,411 was billed for 41 fine-tuning jobs, including jobs that were stopped or could not be served and therefore do not appear in Appendix B.

The data and results in the package are released under CC-BY-SA-4.0 because they contain material derived from FiNER-139. Banking77 and LEDGAR are used under CC-BY-4.0, the CFPB Consumer Complaint Database is CC0 and the National Vulnerability Database is in the public domain. The scripts are released under the MIT License.

Competing interests

The authors are affiliated with UltraSafe AI, which develops and sells the RunBios platform evaluated in this report. To allow independent verification, the test sets, all model outputs, the training records and a script that recomputes every reported figure are publicly released. The frontier baselines were queried through the same platform with the same instructions and answer format as the fine-tuned models. This work received no external funding.

Use of AI assistance

An AI assistant was used to help write the analysis and reproduction scripts and to draft parts of the manuscript. The authors reviewed and edited all text, verified every reported number against the released data using the included verification script, and take full responsibility for the content of this report.

Contributions and Acknowledgments

Authors: Arpit Sharma, Tathagat Sharma

Contributors: Bhavesh Soni, Mohit Kumar, Pranjal Bawa, Anubhav Tripathi, Suraj Sharma, Ankit Sharma

We thank the UltraSafe AI team for building and operating the RunBios platform on which this work was carried out.

References

1. Casanueva, I., Temčinas, T., Gerz, D., Henderson, M. and Vulić, I. Efficient Intent Detection with Dual Sentence Encoders. In Proceedings of the 2nd Workshop on Natural Language Processing for Conversational AI, Association for Computational Linguistics, 2020. https://doi.org/10.18653/v1/2020.nlp4convai-1.5
2. Tuggener, D., von Däniken, P., Peetz, T. and Cieliebak, M. LEDGAR: A Large-Scale Multi-label Corpus for Text Classification of Legal Provisions in Contracts. In Proceedings of the Twelfth Language Resources and Evaluation Conference (LREC), pages 1235 to 1241, 2020. https://aclanthology.org/2020.lrec-1.155
3. Chalkidis, I., Jana, A., Hartung, D., Bommarito, M., Androutsopoulos, I., Katz, D. and Aletras, N. LexGLUE: A Benchmark Dataset for Legal Language Understanding in English. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2022. https://aclanthology.org/2022.acl-long.297

4. Loukas, L., Fergadiotis, M., Chalkidis, I., Spyropoulou, E., Malakasiotis, P., Androutsopoulos, I. and Paliouras, G. FiNER: Financial Numeric Entity Recognition for XBRL Tagging. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 4419 to 4431, 2022. https://doi.org/10.18653/v1/2022.acl-long.303
5. Consumer Financial Protection Bureau. Consumer Complaint Database. https://www.consumerfinance.gov/data-research/consumer-complaints/
6. National Institute of Standards and Technology. National Vulnerability Database. https://nvd.nist.gov/
7. The MITRE Corporation. Common Weakness Enumeration, CWE-1000 Research Concepts view. https://cwe.mitre.org/
8. Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L. and Chen, W. LoRA: Low-Rank Adaptation of Large Language Models. In International Conference on Learning Representations (ICLR), 2022. https://arxiv.org/abs/2106.09685
9. McNemar, Q. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. Psychometrika 12(2), pages 153 to 157, 1947. https://doi.org/10.1007/BF02295996
10. Wilson, E. B. Probable Inference, the Law of Succession, and Statistical Inference. Journal of the American Statistical Association 22(158), pages 209 to 212, 1927. https://doi.org/10.1080/01621459.1927.10502953
11. Gururangan, S., Marasović, A., Swayamdipta, S., Lo, K., Beltagy, I., Downey, D. and Smith, N. A. Don't Stop Pretraining: Adapt Language Models to Domains and Tasks. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pages 8342 to 8360, 2020. https://doi.org/10.18653/v1/2020.acl-main.740
12. Scialom, T., Chakrabarty, T. and Muresan, S. Fine-tuned Language Models are Continual Learners. In Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, pages 6107 to 6122, 2022. https://doi.org/10.18653/v1/2022.emnlp-main.410
13. Luo, Y., Yang, Z., Meng, F., Li, Y., Zhou, J. and Zhang, Y. An Empirical Study of Catastrophic Forgetting in Large Language Models During Continual Fine-tuning. arXiv preprint, 2023. https://arxiv.org/abs/2308.08747
14. Wu, T., Luo, L., Li, Y.-F., Pan, S., Vu, T.-T. and Haffari, G. Continual Learning for Large Language Models: A Survey. arXiv preprint, 2024. https://arxiv.org/abs/2402.01364
15. Biderman, D., Portes, J., Gonzalez Ortiz, J. J., Paul, M., Greengard, P., Jennings, C., King, D., Havens, S., Chiley, V., Frankle, J., Blakeney, C. and Cunningham, J. P. LoRA Learns Less and Forgets Less. Transactions on Machine Learning Research, 2024. https://arxiv.org/abs/2405.09673
16. Dettmers, T., Pagnoni, A., Holtzman, A. and Zettlemoyer, L. QLoRA: Efficient Finetuning of Quantized LLMs. In Advances in Neural Information Processing Systems 36 (NeurIPS), 2023. https://arxiv.org/abs/2305.14314
17. Bucher, M. J. J. and Martini, M. Fine-Tuned 'Small' LLMs (Still) Significantly Outperform Zero-Shot Generative AI Models in Text Classification. arXiv preprint, 2024. https://arxiv.org/abs/2406.08660
18. Edwards, A. and Camacho-Collados, J. Language Models for Text Classification: Is In-Context Learning Enough? In Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING), pages 10058 to 10072, 2024. https://aclanthology.org/2024.lrec-main.879
19. Zhao, J., Wang, T., Abid, W., Angus, G., Garg, A., Kinnison, J., Sherstinsky, A., Molino, P., Addair, T. and Rishi, D. LoRA Land: 310 Fine-tuned LLMs that Rival GPT-4, A Technical Report. arXiv preprint, 2024. https://arxiv.org/abs/2405.00732
20. Chen, L., Zaharia, M. and Zou, J. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv preprint, 2023. https://arxiv.org/abs/2305.05176
21. Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., Kadous, M. W. and Stoica, I. RouteLLM: Learning to Route LLMs with Preference Data. arXiv preprint, 2024. https://arxiv.org/abs/2406.18665
22. Dror, R., Baumer, G., Shlomov, S. and Reichart, R. The Hitchhiker's Guide to Testing Statistical Significance in Natural Language Processing. In Proceedings of the 56th Annual Meeting of the Association for

Computational Linguistics (Volume 1: Long Papers), pages 1383 to 1392, 2018. https://doi.org/10.18653/v1/P18-1128

23. Card, D., Henderson, P., Khandelwal, U., Jia, R., Mahowald, K. and Jurafsky, D. With Little Power Comes Great Responsibility. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, pages 9263 to 9274, 2020. https://doi.org/10.18653/v1/2020.emnlp-main.745
24. Miller, E. Adding Error Bars to Evals: A Statistical Approach to Language Model Evaluations. arXiv preprint, 2024. https://arxiv.org/abs/2411.00640
25. Paleyes, A., Urma, R.-G. and Lawrence, N. D. Challenges in Deploying Machine Learning: A Survey of Case Studies. ACM Computing Surveys 55(6), article 114, 2022. https://doi.org/10.1145/3533378

Appendix A. Development paths

For each task, the figure and table show the versions on the path to the selected model, ordered by model family and training-set size.

A.1 Banking intent routing

Assign a retail-banking customer message to one of 77 support intents. The selected model is RunBios fine-tuned (Llama-3.1-8B). It reached 95.0% exact-match accuracy on the 500 test cases against 88.4% for the strongest frontier baseline (Claude Opus 5.5). On the same cases the selected model was correct where Claude Opus 5.5 was wrong 40 times and the reverse occurred 7 times (exact McNemar $p < 0.001$).

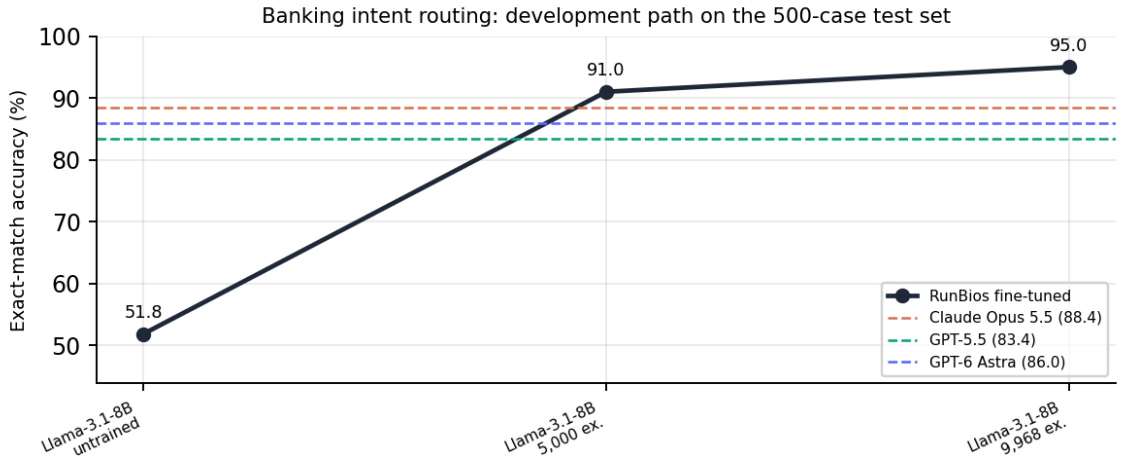


Figure 4. Development path for banking intent routing, ordered by model family and training-set size and ending at the model selected on validation. Dashed lines are the frontier baselines on the same 500 cases.

Table 5. Models on the development path for banking intent routing. Validation and test values are exact-match accuracy in percent.

Model	Training examples	Prompt	Validation	Test	Training cost	Job ID
RunBios fine-tuned (Llama-3.1-8B)	5,000	instruction with label list	-	91.0	$3.80	`ad6b3b4b`
RunBios fine-tuned (Llama-3.1-8B)	9,968	instruction with label list	-	95.0	$10.69	`65fab674`

A.2 Consumer-complaint triage

Assign a consumer-finance complaint narrative to its product and issue category under the regulator's current taxonomy (45 categories). The selected model is RunBios fine-tuned (Llama-3.1-8B). It reached 65.4% exact-match accuracy on the 500 test cases against 56.8% for the strongest frontier baseline (Claude Opus 5.5). On the same cases the selected model was correct where Claude Opus 5.5 was wrong 83 times and the reverse occurred 40 times (exact McNemar $p < 0.001$).

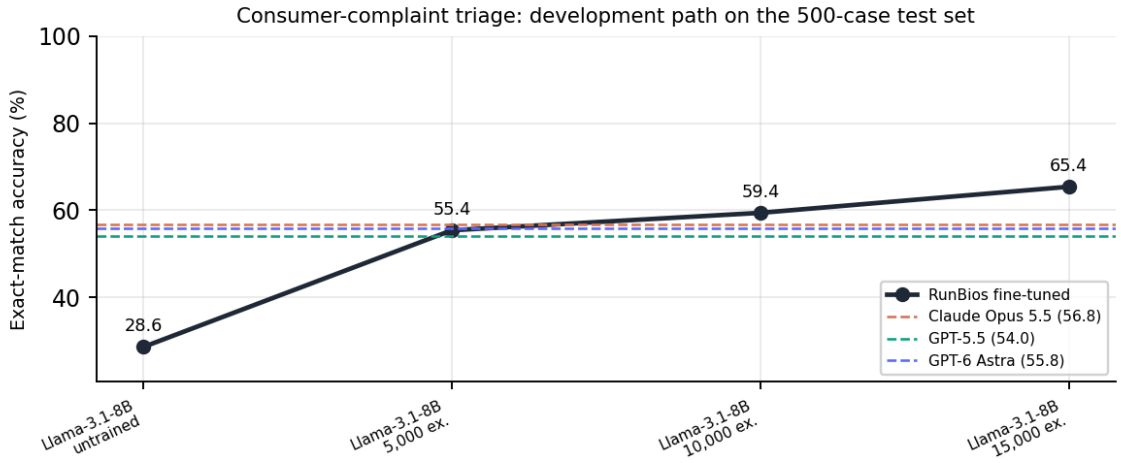


Figure 5. Development path for consumer-complaint triage, ordered by model family and training-set size and ending at the model selected on validation. Dashed lines are the frontier baselines on the same 500 cases.

Table 6. Models on the development path for consumer-complaint triage. Validation and test values are exact-match accuracy in percent.

Model	Training examples	Prompt	Validation	Test	Training cost	Job ID
RunBios fine-tuned (Llama-3.1-8B)	5,000	instruction with label list	-	55.4	$5.82	`91682383`
RunBios fine-tuned (Llama-3.1-8B)	10,000	instruction with label list	-	59.4	$10.48	`757f62d7`
RunBios fine-tuned (Llama-3.1-8B)	15,000	instruction with label list	-	65.4	$17.98	`cacd5204`

A.3 Contract clause classification

Assign a provision from a contract filed with the US SEC to one of 100 clause types. The selected model is RunBios fine-tuned (Qwen3.6-27B). It reached 89.0% exact-match accuracy on the 500 test cases against 84.8% for the strongest frontier baseline (Claude Opus 5.5). On the same cases the selected model was correct where Claude Opus 5.5 was wrong 35 times and the reverse occurred 14 times (exact McNemar $p = 0.004$).

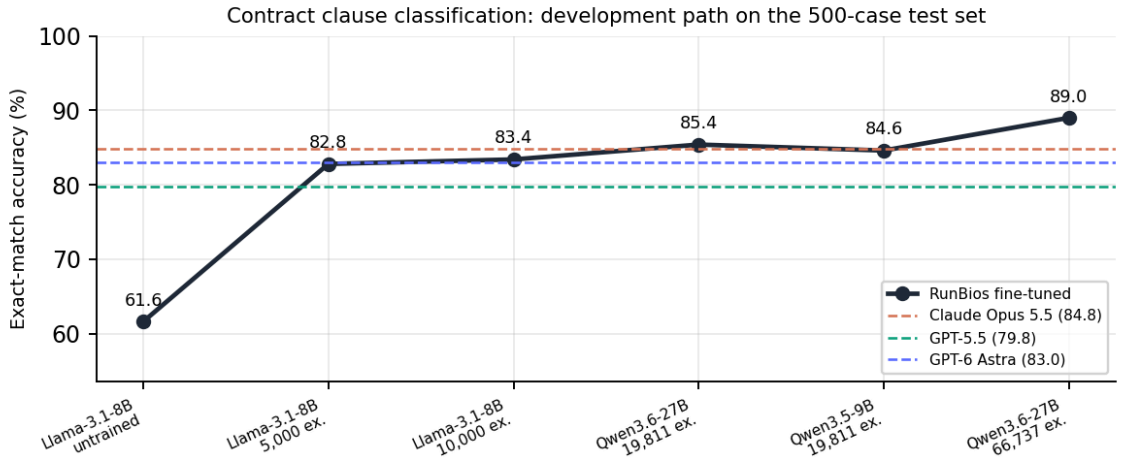


Figure 6. Development path for contract clause classification, ordered by model family and training-set size and ending at the model selected on validation. Dashed lines are the frontier baselines on the same 500 cases.

Table 7. Models on the development path for contract clause classification. Validation and test values are exact-match accuracy in percent.

Model	Training examples	Prompt	Validation	Test	Training cost	Job ID
RunBios fine-tuned (Llama-3.1-8B)	5,000	instruction with label list	-	82.8	$3.34	`87132c8a`
RunBios fine-tuned (Llama-3.1-8B)	10,000	instruction with label list	84.7	83.4	$5.35	`fe457926`
RunBios fine-tuned (Qwen3.6-27B)	19,811	instruction with label list	89.3	85.4	$77.73	`8570ccf5`
RunBios fine-tuned (Qwen3.5-9B)	19,811	instruction with label list	87.3	84.6	$11.52	`af431527`
RunBios fine-tuned (Qwen3.6-27B)	66,737	one-sentence instruction	90.3	89.0	$72.53	`ab3dc81f`

A.4 XBRL numeric tagging

Assign the US-GAAP XBRL tag of one marked numeric value in a sentence from an SEC 10-K or 10-Q filing (139 tags). The selected model is RunBios fine-tuned (Qwen3.5-9B). It reached 87.4% exact-match accuracy on the 500 test cases against 86.4% for the strongest frontier baseline (Claude Opus 5.5). On the same cases the selected model was correct where Claude Opus 5.5 was wrong 25 times and the reverse occurred 20 times (exact McNemar $p = 0.551$).

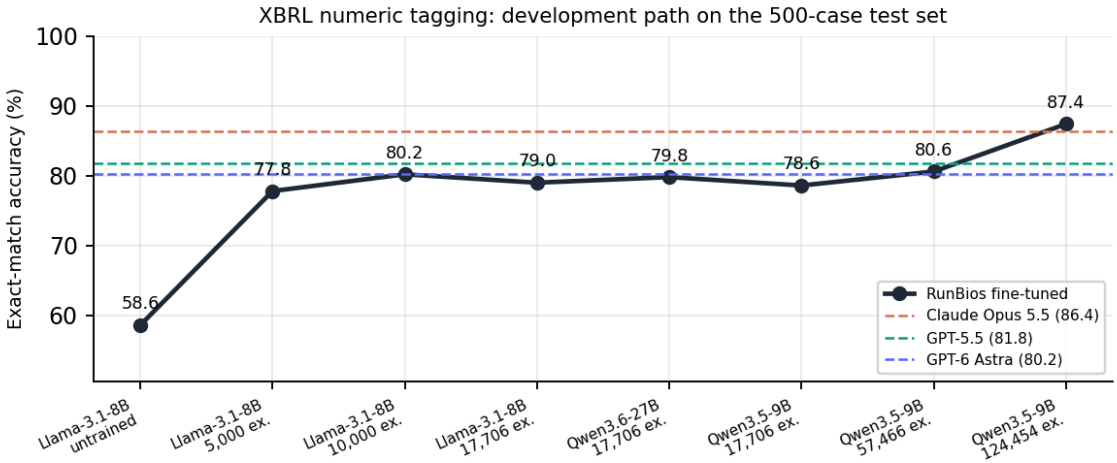


Figure 7. Development path for XBRL numeric tagging, ordered by model family and training-set size and ending at the model selected on validation. Dashed lines are the frontier baselines on the same 500 cases.

Table 8. Models on the development path for XBRL numeric tagging. Validation and test values are exact-match accuracy in percent.

Model	Training examples	Prompt	Validation	Test	Training cost	Job ID
RunBios fine-tuned (Llama-3.1-8B)	5,000	instruction with label list	-	77.8	$8.16	`37494321`
RunBios fine-tuned (Llama-3.1-8B)	10,000	instruction with label list	-	80.2	$20.50	`b03a8b80`
RunBios fine-tuned (Llama-3.1-8B)	17,706	instruction with label list	84.2	79.0	$39.69	`73300605`
RunBios fine-tuned (Qwen3.6-27B)	17,706	instruction with label list	84.5	79.8	$178.03	`3db3afc6`
RunBios fine-tuned (Qwen3.5-9B)	17,706	instruction with label list	85.9	78.6	$39.37	`2751b01b`
RunBios fine-tuned (Qwen3.5-9B)	57,466	instruction with label list	86.5	80.6	$66.54	`18521489`
RunBios fine-tuned (Qwen3.5-9B)	124,454	instruction with label list	91.9	87.4	$192.86	`8cdfab7f`

A.5 Vulnerability root-cause triage

Assign the CWE weakness type of a newly published CVE description (54 weakness types). The selected model is RunBios fine-tuned (Qwen3.6-27B). It reached 76.6% exact-match accuracy on the 500 test cases against 76.2% for the strongest frontier baseline (Claude Opus 5.5). On the same cases the selected model was correct where Claude Opus 5.5 was wrong 34 times and the reverse occurred 32 times (exact McNemar $p = 0.902$).

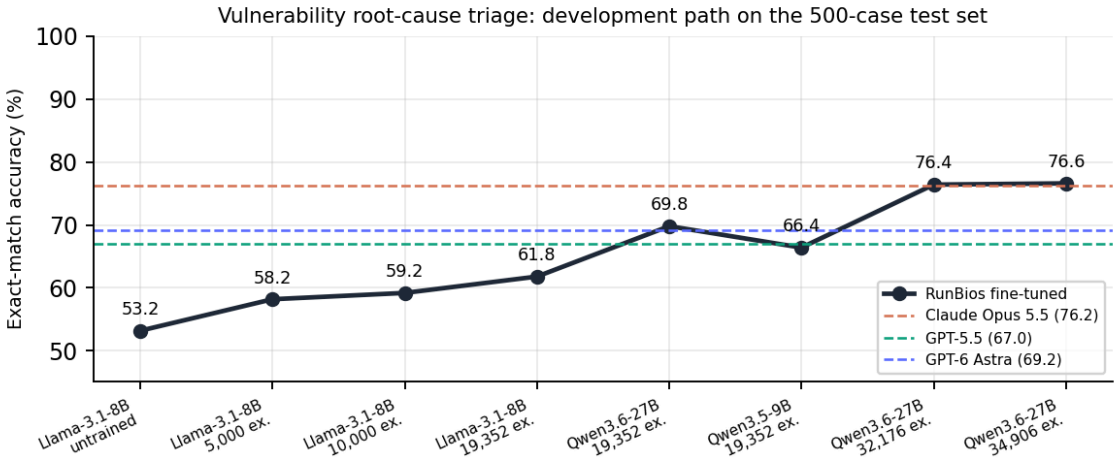


Figure 8. Development path for vulnerability root-cause triage, ordered by model family and training-set size and ending at the model selected on validation. Dashed lines are the frontier baselines on the same 500 cases.

Table 9. Models on the development path for vulnerability root-cause triage. Validation and test values are exact-match accuracy in percent.

Model	Training examples	Prompt	Validation	Test	Training cost	Job ID
RunBios fine-tuned (Llama-3.1-8B)	5,000	instruction with label list	-	58.2	$5.56	`03be31ff`
RunBios fine-tuned (Llama-3.1-8B)	10,000	instruction with label list	-	59.2	$7.46	`fb98a631`
RunBios fine-tuned (Llama-3.1-8B)	19,352	instruction with label list	60.3	61.8	$16.27	`1aed625f`
RunBios fine-tuned (Qwen3.6-27B)	19,352	instruction with label list	69.7	69.8	$136.30	`82aa9ed0`
RunBios fine-tuned (Qwen3.5-9B)	19,352	instruction with label list	67.3	66.4	$20.45	`d6198b3a`
RunBios fine-tuned (Qwen3.6-27B)	32,176	instruction with label list	76.0	76.4	$191.70	`442640c5`
RunBios fine-tuned (Qwen3.6-27B)	34,906	instruction with label list	76.3	76.6	$302.88	`caf60a4a`

Appendix B. Complete run log

For audit, Table 10 lists every specialized configuration that was trained, deployed and scored in Study A, including those not selected. The selected model of each task is the one with the highest validation accuracy.

Table 10. All scored configurations of Study A. Accuracies in percent.

Task	Model	Training examples	Prompt	Validation	Test	
Banking intent routing	RunBios fine-tuned (Llama-3.1-8B)	5,000	label list	-	91.0	
Banking intent routing	RunBios fine-tuned (Llama-3.1-8B)	9,968	label list	-	95.0	selected
Consumer-complaint triage	RunBios fine-tuned (Llama-3.1-8B)	5,000	label list	-	55.4	

Consumer-complaint triage	RunBios fine-tuned (Llama-3.1-8B)	10,000	label list	-	59.4	
Consumer-complaint triage	RunBios fine-tuned (Llama-3.1-8B)	15,000	label list	-	65.4	selected
Contract clause classification	RunBios fine-tuned (Llama-3.1-8B)	5,000	label list	-	82.8	
Contract clause classification	RunBios fine-tuned (Llama-3.1-8B)	10,000	label list	84.7	83.4	
Contract clause classification	RunBios fine-tuned (Qwen3.6-27B)	19,811	label list	89.3	85.4	
Contract clause classification	RunBios fine-tuned (Qwen3.5-9B)	19,811	label list	87.3	84.6	
Contract clause classification	RunBios fine-tuned (Qwen3.6-27B)	66,737	one sentence	90.3	89.0	selected
XBRL numeric tagging	RunBios fine-tuned (Llama-3.1-8B)	5,000	label list	-	77.8	
XBRL numeric tagging	RunBios fine-tuned (Llama-3.1-8B)	10,000	label list	-	80.2	
XBRL numeric tagging	RunBios fine-tuned (Llama-3.1-8B)	17,706	label list	84.2	79.0	
XBRL numeric tagging	RunBios fine-tuned (Qwen3.6-27B)	17,706	label list	84.5	79.8	
XBRL numeric tagging	RunBios fine-tuned (Qwen3.5-9B)	17,706	label list	85.9	78.6	
XBRL numeric tagging	RunBios fine-tuned (Qwen3.5-9B)	57,466	label list	86.5	80.6	
XBRL numeric tagging	RunBios fine-tuned (Qwen3.5-9B)	124,454	label list	91.9	87.4	selected
XBRL numeric tagging	RunBios fine-tuned (Qwen3.5-9B)	230,982	one sentence	89.6	86.4	
XBRL numeric tagging	RunBios fine-tuned (Qwen3.6-27B)	230,982	one sentence	89.6	87.6	
Vulnerability root-cause triage	RunBios fine-tuned (Llama-3.1-8B)	5,000	label list	-	58.2	
Vulnerability root-cause triage	RunBios fine-tuned (Llama-3.1-8B)	10,000	label list	-	59.2	
Vulnerability root-cause triage	RunBios fine-tuned (Llama-3.1-8B)	19,352	label list	60.3	61.8	
Vulnerability root-cause triage	RunBios fine-tuned (Qwen3.6-27B)	19,352	label list	69.7	69.8	
Vulnerability root-cause triage	RunBios fine-tuned (Qwen3.5-9B)	19,352	label list	67.3	66.4	
Vulnerability root-cause triage	RunBios fine-tuned (Qwen3.6-27B)	32,176	label list	76.0	76.4	
Vulnerability root-cause triage	RunBios fine-tuned (Qwen3.6-27B)	34,906	label list	76.3	76.6	selected
Vulnerability root-cause triage	RunBios fine-tuned (Qwen3.6-35B-A3B)	34,906	label list	72.3	74.6	
Vulnerability root-cause triage	RunBios fine-tuned (Qwen3.6-27B)	51,336	one sentence	74.7	75.8	

Vulnerability root-cause triage	RunBios fine-tuned (Qwen3.6-27B)	51,528	label list	74.3	76.4
Vulnerability root-cause triage	RunBios fine-tuned (Qwen3.5-9B)	59,327	label list	68.0	70.4
Vulnerability root-cause triage	RunBios fine-tuned (Qwen3.6-27B)	59,327	label list	70.7	73.0
Vulnerability root-cause triage	RunBios fine-tuned (Qwen3.6-27B)	99,327	label list	72.0	71.4